

Part 1:

Row Key & Sharding:

I would use the row key salt#city#make#modelyear#dol_vehicle_id. The salt prefix is able to spread records across multiple tablets, which reduces hotspotting, while the rest of the key keeps related EV records logically grouped. This improves distributed write scalability, although queries by only City or only Make are less direct because matching rows are spread across multiple salted key ranges.

Column Families:

I would organize the data into four column families: identity (DOL Vehicle ID, VIN (1-10)), vehicle (Make, Model, Model Year, Electric Vehicle Type, CAFV Eligibility), location (County, City, State, Postal Code, Legislative District, Vehicle Location, 2020 Census Tract), and utility (Electric Range, Electric Utility). I believe this helps improve distributed read performance because related attributes are stored together, and queries can read only the relevant families.

Horizontal Scaling:

A hashed shard key on DOL Vehicle ID would distribute writes more evenly and avoid hotspotting, making it better for cluster balance and scalability. A range-based shard key on Model Year would be better for year-based analytical queries because related records stay closer together. However, this key could create uneven shard load if recent years overpower the workload.

Part 4:

I measured the performance of the two write endpoints by sending 50 requests to each. The average latency for `/insert-fast` was 0.00539 seconds, and the average latency for `/insert-safe` was 0.02229 seconds. This shows that `/insert-safe` is slower because it uses `WriteConcern(w="majority")`, which waits for replication to a majority of replica set members before acknowledging the write. On the other hand, `/insert-fast` uses `WriteConcern(w=1)`, which only waits for acknowledgment from the primary node.

Based on the CAP theorem, distributed systems must balance consistency, availability, and partition tolerance. In this assignment, `w=1` favors lower latency and higher availability from the application's perspective, while `w="majority"` favors stronger consistency and durability. A real-world scenario where I would intentionally choose the faster primary-only write is user interaction logging in a large-scale web application, such as tracking clicks, page views, or ad impressions. For example, when a user clicks on an advertisement or navigates between pages, the system records these events for analytics and recommendation systems. In this case, it is more important to handle a very high volume of events with low latency than to guarantee that every single event is perfectly durable. Even if a small number of events are lost due to a primary node failure before replication, the overall analytics results will not be significantly affected.